

Zadi Tech System Implementation Documentation

Last reviewed locally: 2026-07-15

1. Purpose

This project is a Laravel 12 business operations system for Zadi Tech. It combines CRM, sales documents, invoicing, receipts, POS orders, projects, procurement, warranties, letters, finance, cost accounting, administration, and client portal workflows in one web application.

The application is implemented as a traditional server-rendered Laravel monolith:

- Laravel routes in `routes/web.php`
- Controllers in `app/Http/Controllers`
- Eloquent models in `app/Models`
- Domain services in `app/Services`
- Blade views in `resources/views`
- Vite and Tailwind assets in `resources/css` and `resources/js`
- Database schema in `database/migrations`

2. Runtime Stack

- PHP: `^8.2`
- Framework: Laravel `^12.0`
- Database: MySQL by default, configured through `.env`
- PDF generation: `barryvdh/laravel-dompdf`
- QR codes: `endroid/qr-code`
- Frontend build: Vite 7, Tailwind CSS 4, Laravel Vite plugin
- Session, cache, queue: database-backed in the default `.env.example`

Important Composer packages:

- `laravel/framework`
- `laravel/tinker`
- `barryvdh/laravel-dompdf`
- `endroid/qr-code`

Important NPM packages:

- `vite`
- `laravel-vite-plugin`
- `tailwindcss`
- `@tailwindcss/vite`
- `axios`
- `concurrently`

3. Local Setup

From the project root:

```
composer install
copy .env.example .env
php artisan key:generate
php artisan migrate --force
npm install
npm run build
php artisan serve
```

This checkout included `vendor.zip`, so `vendor/` can also be restored by extracting that archive when Composer dependencies are missing.

The local `.env` currently points to:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=zadi_tech
DB_USERNAME=root
DB_PASSWORD=
```

Default seeded admin from `database/seeders/DatabaseSeeder.php` :

```
Email: admin@zaditech.com
Username: ZachariaM
Password: Zach@123
Role: admin
```

4. Application Entry Points

Public routes:

- `/` redirects to `/dashboard`
- `/login` renders the login page
- `/invoice/{token}` shows a public invoice
- `/invoice/{token}/download` downloads a public invoice PDF
- `/track-order/{key}` shows public POS order tracking
- `/verify/letter/{letter}` verifies a public letter
- `/portal/activate/{token}` activates a client portal user
- `/activate/{token}` and `/account-recovery/{token}` handle internal user activation and recovery

Authenticated routes:

- `/dashboard`
- `/profile`
- `/portal`
- `/businesses`
- `/administration`
- `/clients`
- `/projects`
- `/quotations`
- `/invoices`

- `/receipts`
- `/pos-orders`
- `/products`
- `/letters`
- `/finance`
- `/accounting`
- `/erp/*`
- `/settings`

Most business operations sit inside:

```
Route::middleware('auth')->group(...)
Route::middleware('admin')->group(...)
```

The `admin` middleware does not mean only `role = admin`; it blocks only `client_portal` users from admin surfaces.

5. Authentication and Access Control

Authentication is handled by `App\Http\Controllers\AuthController`.

Supported login methods:

- Password login with username or email
- OTP login through `otp_codes`
- Magic link login through `login_tokens`
- Password reset using Laravel password broker

The login flow checks:

- Whether the user exists by email or username
- Whether the account is active
- Whether the account status is `Active`
- Whether password login is enabled for that user
- Whether the user is temporarily locked

Security-related tables and fields are introduced by later migrations:

- `failed_login_attempts`
- `locked_at`
- `last_login_at`
- `last_login_ip`
- `security_settings`
- `login_activities`
- `user_devices`
- `password_histories`

IAM is implemented by `App\Services\IamService` and `RequirePermission` middleware.

IAM concepts:

- `iam_permissions`
- `iam_roles`
- `iam_permission_role`

- `iam_permission_user`
- `business_user`
- Direct permissions plus role permissions are merged per user.

System roles include:

- System Administrator
- Business Administrator
- Finance Manager
- Accountant
- Procurement Officer
- Project Manager
- Technician
- Sales Executive
- HR Manager
- Operations Manager
- Store Manager
- Director
- Viewer

Permission examples:

- `administration.view`
- `users.create`
- `finance.gl.unreverse`
- `letters.create`
- `reports.export`

6. Business Scoping

The system supports multiple businesses in one database.

Core implementation:

- `App\Models\Business`
- `App\Support\ActiveBusiness`
- `App\Models\Concerns\BelongsToBusiness`
- `businesses` table
- `business_user` pivot table

`ActiveBusiness` stores the active business in session using:

```
active_business_id
```

If no business exists, `ActiveBusiness::ensureDefaults()` creates:

- Zadi Tech
- ZADI INTERIORS

Most business-owned models use the `BelongsToBusiness` trait. That trait:

- Adds a global Eloquent scope for `business_id = ActiveBusiness::id()`
- Auto-fills `business_id` when creating records

Public document and tracking routes intentionally bypass the business global scope when loading shared records by public token or tracking key.

7. Main Modules

7.1 Dashboard

Implemented in:

- `app/Http/Controllers/DashboardController.php`
- `resources/views/dashboard.blade.php`

The dashboard aggregates:

- Invoice count
- Quotation count
- Receipt count
- POS order count
- Paid and unpaid invoice counts
- Pending payments
- POS revenue
- Product count
- Project count when project support exists
- Receivables, collected amount, profit, tax due, and supplier due when ERP tables exist

It also builds performance chart data for daily, weekly, monthly, yearly, and custom date windows.

7.2 Company Settings

Implemented in:

- `CompanySettingsController`
- `CompanySetting`
- `PaymentMethod`
- `TermsCondition`
- `Signatory`

This module stores:

- Company name, logo, address, phone, email, website
- Tax name and tax rate
- Currency and locale
- Default terms
- Payment methods
- Terms and conditions
- Signatories and signatures

These settings are reused in PDF generation, receipts, invoices, letters, and public document pages.

7.3 Clients, Contacts, Sites, and Projects

Implemented in:

- `ClientController`

- `ProjectController`
- `ClientMergeService`
- `Client`, `Contact`, `Site`, `Project`

Clients can be companies or individuals. For company clients, the system supports:

- Multiple contacts
- A primary contact
- Sites
- Projects

Projects link to:

- Client
- Site
- Contact
- Quotations
- Invoices
- Project costs and expenses
- Supplier quotes
- Purchase orders
- Supplier invoices
- Warranties
- Documents
- Handover records
- Letters
- Cost center

The `Project` model exposes business calculations:

- `expectedCost()`
- `actualCost()`
- `revenue()`
- `collected()`

7.4 Quotations

Implemented in:

- `QuotationController`
- `Quotation`
- `QuotationItem`
- `DocumentService`
- `resources/views/quotations/*`
- `resources/views/pdf/document.blade.php`

Quotation workflow:

1. Create quotation for an existing or new client.
2. Add line items with quantity, unit price, discount, and tax.
3. `DocumentService` calculates subtotal, discount, tax, and total.
4. A quotation number is generated as `QT-YYYY-0001`.

5. The quotation can be downloaded as PDF or emailed.
6. A quotation can be converted into an invoice.

7.5 Invoices, Payments, and Receipts

Implemented in:

- InvoiceController
- ReceiptController
- Invoice
- InvoiceItem
- Payment
- Receipt
- InvoiceAllocation
- ReceiptAllocation
- InvoicePartPaymentService
- InvoicePosOrderService
- InvoiceVerificationService

Invoice workflow:

1. Create invoice for an existing or new client.
2. Add line items.
3. Generate invoice number as INV-YYYY-0001 .
4. Generate public token for public sharing.
5. Calculate totals and balance.
6. Download or email PDF.
7. Record payments against source invoices.
8. Generate receipts automatically when payments are recorded.

Payment workflow:

1. Payment is stored under the invoice.
2. Invoice amount_paid , balance , and payment_status are recalculated.
3. Receipt number is generated as RCT-YYYY-0001 .
4. Optional receipt allocations are stored when allocation tables exist.
5. POS order sync is triggered when the invoice is linked to POS orders.

Advanced invoice features include:

- Part payment invoices
- Stage payment invoices
- VAT-only invoices
- Balance invoices
- Combined allocation invoices

The system guards some features with schema checks, so advanced flows only appear once their migrations have run.

7.6 POS Orders and Products

Implemented in:

- PosOrderController

- ProductController
- Product
- ProductCategory
- PosOrder
- PosOrderItem
- PosOrderPayment

Product management supports:

- Categories
- SKU
- Price
- Cost price
- Stock quantity
- Active/inactive state

POS order workflow:

1. Create order manually or import from CSV.
2. Add products or custom line items.
3. Generate order number as POS-YYYY-0001 .
4. Generate tracking key for public tracking.
5. Store customer information.
6. Store payments and order status.
7. Approve, cancel, update, or delete orders.

CSV import normalizes common sales platform column names into:

- Date
- Order number
- Status
- Customer
- Customer type
- Products
- Quantity
- Net sales

7.7 Letters and Correspondence

Implemented in:

- LetterController
- LetterService
- Letter
- LetterTemplate
- LetterAttachment
- LetterVersion
- DocumentMedia
- resources/views/letters/*
- resources/views/pdf/letter.blade.php

The Letters module is isolated from the existing invoice, quotation, and receipt PDF generation.

Letter workflow:

1. Create a letter directly or from linked records.
2. Choose a template or start with custom content.
3. Render placeholders from client, site, project, invoice, receipt, payment, warranty, company, and signatory context.
4. Generate letter number as `LTR-YYYY-0001`, or with a custom prefix.
5. Save versions whenever content/status changes.
6. Submit, approve, archive, deliver, preview, download PDF, or download DOCX.
7. Publish to portal or send by email.
8. Verify publicly through QR-code-backed verification route.

Linked letter entry points:

- Invoice balance letter
- Receipt acknowledgement letter
- Project handover letter
- Warranty letter

Template categories include:

- Financial
- Projects
- Legal and Contracts
- Warranty and Support
- General Business
- Procurement

The letter editor stores HTML content for rich formatting and PDF compatibility. Image uploads go through:

```
POST /letters/images
```

Uploaded images are stored under:

```
storage/app/public/letters/images
```

When the `document_media` table exists, uploads are tracked there.

7.8 ERP Operations

Implemented in:

- `ErpController`
- `ProjectCost`
- `ProjectExpense`
- `ProjectDocument`
- `DocumentTemplate`
- `Supplier`
- `SupplierQuote`
- `PurchaseOrder`
- `GoodsReceivedNote`
- `SupplierInvoice`

- `SupplierPayment`
- `Warranty`
- `WarrantyClaim`
- `ClientPortalInvitation`
- `BusinessTemplate`

ERP surfaces:

- Profit
- Procurement
- Warranties
- Client portal management
- Templates
- Reports

Supported workflows:

- Project costs
- Project expenses
- Project documents
- Project handover checklists
- Suppliers
- Supplier quotes
- Purchase orders
- Goods received notes
- Supplier invoices
- Supplier payments
- Warranty registration
- Warranty claims
- Client portal invitations
- Business templates
- User authentication settings

7.9 Finance

Implemented in:

- `FinanceController`
- `FinanceService`
- `FinanceAccount`
- `JournalEntry`
- `JournalLine`
- `BankAccount`
- `BankTransaction`
- `FixedAsset`
- `AssetDepreciationSchedule`

Finance features:

- Chart of accounts
- Balanced journal entries

- Journal reversal
- Restricted journal unreverse permission
- Syncing legacy invoices, payments, supplier bills, and supplier payments into journals
- Bank and cash accounts
- Bank transactions
- Reconciliation
- Fixed assets
- Depreciation schedules
- Financial period closing
- Tax record tables
- AR and AP aging
- Income, expense, asset, liability, and equity reports

`FinanceService::post()` enforces:

- Locked periods cannot receive entries.
- Debits and credits must balance.
- Source documents are only posted once per `source_type` and `source_id`.

Default chart of accounts is seeded from `FinanceService::ACCOUNTS`.

7.10 Cost Accounting

Implemented in:

- `CostAccountingController`
- `CostAccountingService`
- `Department`
- `CostCenter`
- `AccountingBudget`
- `AccountingAllocation`
- `BudgetAlert`
- `AccountingAuditLog`

Cost accounting supports:

- Departments
- Cost centers
- Project cost centers
- Budgets
- Budget approval
- Manual allocations
- Industry-based starter structures
- Budget threshold alerts
- Project, department, and cost-center reports

Industry presets include:

- Technology
- Construction
- Retail

- Healthcare
- Manufacturing

Projects can automatically receive cost centers through `CostAccountingService::ensureProjectCostCenter()`.

7.11 Administration

Implemented in:

- `AdministrationController`
- `IamService`
- `AdminAuditLog`
- `Branch`
- `Team`
- `UserInvitation`
- `UserDevice`
- `SecuritySetting`
- `MailSetting`

Administration features:

- User listing
- User creation wizard
- Clone user permissions and teams
- Custom role creation
- Branches
- Teams
- Approval workflows
- User activation links
- Recovery links
- Force password reset
- Account lock/unlock
- Session/device revocation
- Security settings
- Mail settings
- Admin audit logs
- Login activity

Administration routes are wrapped with `permission:administration.view`, and sensitive actions add more specific permissions such as `security.manage`, `users.deactivate`, and `audit.view`.

7.12 Client Portal

Implemented in:

- `PortalController`
- `ClientPortalInvitation`
- `portal/*` Blade views

Portal workflow:

1. Admin creates a portal invitation.

2. Client opens `/portal/activate/{token}` .
3. Client creates a password.
4. User is created with `role = client_portal` .
5. Client logs into `/portal` .
6. Portal shows that client's projects, documents, warranties, and invoices.

Client portal users are redirected away from admin routes by `EnsureAdminAccess` .

8. Database Design Summary

Core Laravel tables:

- `users`
- `password_reset_tokens`
- `sessions`
- `cache`
- `jobs`

Business and settings:

- `businesses`
- `company_settings`
- `payment_methods`
- `terms_conditions`
- `signatories`

CRM/project tables:

- `clients`
- `contacts`
- `sites`
- `projects`

Sales documents:

- `quotations`
- `quotation_items`
- `invoices`
- `invoice_items`
- `payments`
- `receipts`
- `email_logs`
- `invoice_allocations`
- `receipt_allocations`

POS:

- `product_categories`
- `products`
- `pos_orders`
- `pos_order_items`
- `pos_order_payments`

Letters:

- letter_templates
- letters
- letter_attachments
- letter_versions
- document_media
- template_categories

ERP:

- project_costs
- project_expenses
- project_documents
- document_templates
- suppliers
- supplier_quotes
- purchase_orders
- goods_received_notes
- supplier_invoices
- supplier_payments
- warranties
- warranty_claims
- client_portal_invitations
- business_templates

Finance:

- finance_accounts
- finance_periods
- journal_entries
- journal_lines
- bank_accounts
- bank_transactions
- bank_statement_lines
- fixed_assets
- asset_depreciation_schedules
- tax_records

Cost accounting:

- departments
- cost_centers
- accounting_budgets
- accounting_allocations
- budget_alerts
- accounting_audit_logs

Administration/IAM:

- `branches`
- `iam_roles`
- `iam_permissions`
- `iam_permission_role`
- `iam_permission_user`
- `business_user`
- `teams`
- `team_user`
- `user_invitations`
- `login_activities`
- `user_devices`
- `approval_workflows`
- `approval_workflow_steps`
- `approval_requests`
- `approval_actions`
- `admin_audit_logs`
- `password_histories`
- `security_settings`
- `admin_recovery_links`
- `mail_settings`

9. Document Numbering

Document numbers are generated in services with per-business locking.

`DocumentService` generates:

- Quotations: `QT-YYYY-0001`
- Invoices: `INV-YYYY-0001`
- Receipts: `RCT-YYYY-0001`
- POS orders: `POS-YYYY-0001`

`LetterService` generates:

- Letters: `LTR-YYYY-0001`
- Or `{CUSTOMPREFIX}-YYYY-0001`

Both services lock the active business row before calculating the next number. This reduces duplicate numbering risk when multiple users create documents at the same time.

10. PDF and Public Verification

PDF rendering uses DomPDF.

PDF views:

- `resources/views/pdf/document.blade.php`
- `resources/views/pdf/receipt.blade.php`
- `resources/views/pdf/letter.blade.php`
- `resources/views/pdf/project-document.blade.php`

Public invoice sharing uses `public_token`.

Letter verification uses:

- `GET /verify/letter/{letter}`
- QR codes generated by `endroid/qr-code`
- `LetterService::qrCodeDataUri()`

Invoice verification is handled by `InvoiceVerificationService`.

11. Frontend Implementation

The UI is Blade-rendered and uses Bootstrap-like markup patterns in the views. Laravel Vite builds:

- `resources/css/app.css`
- `resources/js/app.js`

Vite config:

```
input: ['resources/css/app.css', 'resources/js/app.js']
```

Tailwind CSS 4 is configured with Blade and JS sources. The current `resources/js/app.js` only imports `bootstrap.js`, so most UI behavior is handled inline in Blade templates or by browser defaults.

12. Auditing

There are two auditing paths:

1. Administration audit through `AdminAuditLog` and `IamService::audit()`.
2. Accounting audit through `AccountingAuditLog` and `CostAccountingService::audit()`.

`RecordMajorActivity` middleware is appended to the web middleware stack. It records major successful POST, PUT, PATCH, and DELETE actions into `admin_audit_logs` when that table exists.

Tracked action names include:

- store
- update
- destroy
- approve
- archive
- convert
- payment
- reconcile
- close
- reverse
- unreverse
- deliver
- submit
- merge
- import
- sync

- acknowledge

13. Current Local Migration Status

As checked locally on 2026-07-15, these migrations were pending in this database:

- 2026_04_27_000001_create_zadi_admin_tables
- 2026_07_04_000001_create_document_management_tables
- 2026_07_13_000001_add_content_type_to_letter_tables
- 2026_07_14_000001_create_cost_accounting_tables
- 2026_07_15_000001_create_finance_tables
- 2026_07_16_000001_complete_cost_accounting_workflows
- 2026_07_17_000001_create_administration_module
- 2026_07_18_000001_enhance_user_provisioning
- 2026_07_19_000001_create_mail_settings_table

Because many controllers use `Schema::hasTable()` and `Schema::hasColumn()` guards, parts of the UI can degrade or hide safely when optional module tables are not installed. For the full system, run all migrations.

14. Important Implementation Patterns

Schema guards

Many features check whether tables or columns exist before using them. This allows the app to run across partially migrated deployments, but it also means missing migrations can make features invisible.

Examples:

- Letters require `letters` and `letter_templates`.
- ERP project tabs require `project_costs`.
- Client company structure requires the later client/site/project tables.
- OTP and magic link require their token tables.
- POS payment history requires `pos_order_payments`.

Business global scopes

Most domain models automatically filter by active business. Public pages bypass global scope only where a public token or tracking key is expected.

Service-based calculations

Reusable calculations are placed in services:

- Totals and numbers: `DocumentService`
- Letters, templates, QR codes, PDFs: `LetterService`
- GL posting, reports, aging: `FinanceService`
- Cost reporting and project cost centers: `CostAccountingService`
- Client merging: `ClientMergeService`
- Invoice part payments and POS sync: invoice services

Server-rendered workflows

The application is not an API-first SPA. Most user actions are form posts to Laravel controllers followed by redirects with flash messages.

15. Operational Commands

Start local server:

```
php artisan serve
```

Run migrations:

```
php artisan migrate --force
```

Check migration status:

```
php artisan migrate:status
```

Seed default data:

```
php artisan db:seed --class=DatabaseSeeder
```

Build frontend assets:

```
npm run build
```

Run tests:

```
php artisan test
```

Clear caches after deployment:

```
php artisan optimize:clear  
php artisan view:clear  
php artisan route:clear  
php artisan config:clear
```

16. Known Local Notes

- The local app is served at `http://127.0.0.1:8000` when `php artisan serve` is running.
- The app key has been generated in `.env`.
- `vendor/` is restored locally.
- `node_modules/` is not committed and should be installed with `npm install` when frontend builds are needed.
- `public/storage` is ignored, so run `php artisan storage:link` if public uploads do not render.
- The default README is still Laravel's starter README; this document is the implementation-specific guide.